# Razonamiento Automático 2026/27 Challenge 3: Enunciado entrenamiento

# Arrays de bytes: base de mini-Tetris

## Primer challenge de equipo

En clase os encontraréis con ejercicios como estos y podréis repartirlos en el equipo y resolverlos como creáis oportuno. Podéis hacer 2 o 3 cada uno, o podéis formar parejas o subequipos para colaborar resolviendo. Podéis hacer un esquema de solución/revisión. Podéis organizaros como mejor veáis. La idea es que el challenge se asigna al equipo y lo resolvéis entre todos.

Todos los ejercicios de programación en este challenge son rutinas. Juntas forman la base de código para implementar un mini-tetris: un tablero como array de bytes, piezas que se comprueban y se dibujan, movimiento con el pad.

No es necesario, ni se pedirá en clase, que juntéis todos los ejercicios y que la implementación final funcione como un tetris. De hecho, es una base, pero no está completo. Eso sí, como ejercicio adicional podéis probar a completarlo y sería un buen ejercicio para practicar.

## Ejercicios y bienes

Este challenge de entrenamiento tiene 21 ejercicios. En clase tendréis una versión ajustada a 12 bienes: no serán los 21 los que tendréis en clase. Eso sí, como siempre, los 12 bienes de clase serán ejercicios como estos, pero con datos distintos.

Lo importante es que sepas hacerlos por ti mismx. Si sabes resolver los ejercicios por ti mismx y sin ayudas, has aprendido lo que necesitas: no tendrás problemas en el challenge.

## Objetivos de aprendizaje

- Arrays de bytes en 1 dimensión (sin structs)
- **La RAM no tiene valores iniciales garantizados** (son aleatorios): contabilizamos bytes con `DS` (*define stream*) para tener reservado lo que necesitamos; cualquier valor de partida (incluido un ***centinela***) hay que escribirlo por código, al arrancar.
- Indexar un array 2D (fila, columna) mediante aritmética simple sobre un array 1D: el array se guarda en ram por filas, como los tilemaps de la VRAM.
- **Extraer un cálculo repetido a su propia rutina** ( `board_cell_address` ) y reutilizarla desde varias otras, en vez de repetirlo
- Recorrer arrays: rellenar, contar, comprobar una condición sobre los elementos
- Recorrido con paso fijo distinto de 1 (para recorrer columnas, por ejemplo)
- Copiar/desplazar datos entre dos zonas de un array
- **Agrupar varios campos relacionados** (mini estructuras) y compararlos
- Lectura de joypad con límites (repaso, con comprobación de bordes)
- Transformar un array en otro con distinto "ancho" (tablero → tilemap)
- **Dos técnicas para saber cuándo terminar un recorrido**:
  - Comparar el puntero contra una dirección de fin calculada
  - Comprobar con un valor especial (centinela) al final de los datos
- Combinar dos comprobaciones sencillas en una sola rutina
- **Devolver resultados en un flag ( `Z` / `NZ` ) cuando el resultado es un sí/no**: así el que llama puede saltar condicional directamente.
- **Codificar una forma como una secuencia de pasos** ( `X,Y,W,H,I,I,...,0` , un dato autocontenido: posición + tamaño + forma en un único bloque) en vez de listar celdas sueltas y separar "cabida" en dos capas independientes (geometría vs. ocupación) que se combinan llamándose entre sí.

## Código común

⚠ **Este código inicial es común y disponible para todos los ejercicios. El código de vuestras soluciones debe usarlo.**

*Código común*

```
DEF BOARD_WIDTH  = 8
DEF BOARD_HEIGHT = 12
DEF BOARD_SIZE   = BOARD_WIDTH * BOARD_HEIGHT   ;; = 96

;; board: 1 byte per cell.
;;  -  0 → empty
;;  - >0 → filled (color/tile)
;; Cell index (row, column) = row * BOARD_WIDTH + column
DEF BOARD_SENTINEL = $FF    ;; Reserved value. Marks end of array.
                            ;; Never use $FF as a real color/tile

SECTION "Board", WRAM0
board: DS BOARD_SIZE   ;; Enough space for the board
board_end:             ;; Address right after the last board cell (Exercise 14)
board_sentinel: DS 1   ;; Centinel right after the last board cell
                       ;; * uninitialized (RAM has no starting value)
                       ;;   must be set in code (Exercise 2)

;; Struct for storing Pieces
;; Definition: X, Y, W, H, I, I, ..., I, 0
;;   X, Y: board position of the piece's first cell
;;   W, H: width and height of the piece's bounding box
;;   I: one "increment" per additional cell (besides the first).
;;      A byte in the format %1111DULR, in negative logic;
;;      A bit=0 → move in that direction: Down, Up, Left, Right
;;      same format as a D-Pad reading (Exercise 12)
;;   0: sentinel, marks the end of the increment list
;;
;; Example — a Z-piece (3x2 bounding box) with its first cell at
;; column=1, row=2, will fit in memory like this:
;;
;;  > address    00 01 02 03 04 05 06 07   (8 bytes)
```

```
;;  > content  | 01 02 03 02 FE F7 FE 00 |
;;  > meaning    X  Y  W  H  I1 I2 I3 S
;;  >                          R  D  R  End
;; (visits board cells (row=2,col=1), (2,2), (3,2), (3,3))
DEF I_RIGHT_BIT = 0
DEF I_LEFT_BIT  = 1
DEF I_UP_BIT    = 2
DEF I_DOWN_BIT  = 3

SECTION "Piece Data", WRAM0
;; (Y,X) struct for locating current piece
piece_row: DS 1   ;; current row of the piece
piece_col: DS 1   ;; current column of the piece being moved with the pad

SECTION "Obstacle Data", WRAM0
;; (Y,X) struct for locating current obstacle
obstacle_row: DS 1   ;; row of a fixed obstacle (for Exercise 6)
obstacle_col: DS 1   ;; column of that obstacle
```

---

## Ejercicio 1

**Implementa** `board_clear`

**Debe poner a 0 todas las celdas del** `board`**.**
Así todo el tablero quedará vacío (situación inicial). El tablero son `BOARD_SIZE` bytes a partir de la dirección `board`.

*board_clear*

```
SECTION "Board Code", ROM0
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Sets the whole board array to 0 (empty board)
;;
board_clear::
    ;;; <<YOUR CODE>>
```

## Ejercicio 2

**Implementa** `board_init`

**La Game Boy arranca con bytes aleatorios en RAM.**

En la Game Boy, la RAM comienza con datos aleatorios, y no podemos ponerles un valor inicial; solo podemos etiquetar y seleccionar lugares y tamaños (con etiquetas y `DS`) para luego escribirlos.

Asume que `board_clear` ya existe y funciona, y úsala para iniciar las celdas del tablero. Después inicializa el centinela `board_sentinel`: también está en RAM, así que hay que establecer su valor inicial por código o se quedará aleatorio.

*board_init*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Initializes everything needed to start:
;; clears the board and sets the sentinel.
;;
board_init::
    ;;; <<YOUR CODE>>
```

## Ejercicio 3

**Implementa** `board_cell_address`

**Cómo acceder a un elemento de un array 2D**

Cuando quieres cambiar un elemento en un array/tabla 2D (como el `board`) necesita saber en qué dirección de memoria está el elemento. Normalmente, lo que tienes son coordenadas (X,Y) (los índices, como en `array[5][7]`) y la etiqueta (dirección de memoria) del inicio del array (`board`).
Esta rutina usa esos datos para calcular la dirección de memoria de un elemento (X,Y) del array 2D `board`. Cada elemento, ocupa un byte, porque sólo es un ID de tile, y se distribuyen en memoria por filas (primero la primera fila entera, luego la 2ª, etc).

**Calcula la dirección de memoria del elemento Y,X (fila, columna).**

**Cómo calcular la dirección de memoria**

Calcula el índice en el array: el `idx` que usarías en `board[idx]`, si fuera un array de 1 dimensión. Con esta fórmula:

`IDX = fila * BOARD_WIDTH + columna` ;; Avanzar Y filas + X columnas

Y ya que tienes una etiqueta `board` que dice dónde empieza el tablero en memoria, sólo tienes que sumar `IDX` a `board` y tendrás la dirección del elemento (byte) que queremos. Devuelve esa dirección en `hl`.

Después usarás esta rutina para acceder al `board`.

**Pista**

Como `BOARD_WIDTH` vale 8, multiplicar por 8 es lo mismo que sumar el número consigo mismo 3 veces (`x2`, `x4`, `x8`)

*board_cell_address*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Computes the memory address of a board cell,
;; given its row and column.
;; 📥 INPUT:
;;    B: row
;;    C: column
;; 🔙 RETURNS:
;;    HL: address of that cell in board
;;
board_cell_address::
    ;;; <<YOUR CODE>>
```

## Ejercicio 4

**Implementa** `board_get_cell` **y** `board_set_cell`

**Acceder a una celda del board**

Ahora que tienes `board_cell_address` (Ejercicio 3), leer o escribir una celda es solo "calcular la dirección y tocar esa dirección". asume que `board_cell_address` ya existe y funciona ( `B=fila, C=columna → HL=dirección` ) y úsala. Te simplificará las rutinas que tienes que implementar ahora.

**Detalle**

Fíjate en que `board_set_cell` recibe el valor a escribir en `E` , no en `A` : `board_cell_address` usa `A` internamente para calcular la dirección, así que si el valor llegara en `A` lo perderías antes de poder usarlo.

*board_get_cell / board_set_cell*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Reads the value of a board cell.
;; 📥 INPUT: B=row, C=column  🔙 RETURNS: A=value of that cell
board_get_cell::
    ;;; <<YOUR CODE>>

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Writes a value into a board cell.
;; 📥 INPUT: B=row, C=column, E=value to write
board_set_cell::
    ;;; <<YOUR CODE>>
```

## Ejercicio 5

**Implementa** `board_row_is_full`

**Filas llenas**

Una fila está "llena" cuando **todas** sus `BOARD_WIDTH` están llenas (todas `!= 0`). Si una celda vale 0, está vacía: si fila ya no estará llena.

Es el primer ejercicio en el que recorres varias celdas seguidas de una fila: necesitas comprobarlas todas antes de poder decir "está llena".

**Dónde comienza una fila**

El inicio de una fila es la celda `(fila, 0)`. Usa `board_cell_address` para obtener esa dirección en vez de recalcularla tú mismo. A partir de ahí, cada nueva celda está justo en la posición siguiente en memoria (+1).

**Devolver resultado usando flags**

Devuelve el resultado en el flag ( `Z` =llena, `NZ` =no llena), no en un registro. Es tan sencillo como hacer una operación que deje el flag así, y volver `ret`. Si nada lo cambia, el flag seguirá así.

Así quien llama a la rutina podrá usarlo directamente para saltar condicionalmente con `jr z` / `jr nz` sin tener que hacer ninguna operación. De paso, liberas registros.

*board_row_is_full*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a board row is full.
;;
;; 📥 INPUT:
;;    B: row
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if full
;;    NZ, if NOT full
;;
board_row_is_full::
    ;;; <<YOUR CODE>>
```

## Ejercicio 6

**Implementa** `piece_touches_obstacle`

**Estructuras de datos simples**

No todo son arrays: `piece_row` / `piece_col` y `obstacle_row` / `obstacle_col` son dos mini estructuras de posición **independientes del tablero**: ningún array aquí, son solo 4 bytes agrupados de dos en dos: cada dos son un par de coordenadas (fila, columna)

**Devuelve el resultado en el flag Z**

- `Z` → coincide (se tocan)
- `NZ` → no coincide (no se tocan)

*piece_touches_obstacle*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether the piece is at the same
;; position as the obstacle.
;;
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if they match (collision)
;;    NZ, if they do NOT match
;;
piece_touches_obstacle::
    ;;; <<YOUR CODE>>
```

## Ejercicio 7

**Implementa** `board_clear_row`

**Limpiar una fila**

Limpiar una fila es rellenar con 0 las `BOARD_WIDTH` celdas de la fila. Servirá para vaciarla, por ejemplo, cuando esté llena (se completa).

**Recuerda**

Usa `board_cell_address` para acceder a la fila que quieres. Usa Columna=0 para el inicio de la fila.

*board_clear_row*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Clears a whole board row.
;;
;; 📥 INPUT:
;;    B: row
;;
board_clear_row::
    ;;; <<YOUR CODE>>
```

## Ejercicio 8

**Implementa** `board_count_filled_in_row`

### Contar celdas ocupadas

Cuenta cuántas celdas de una fila están ocupadas ( `!= 0` ). Parecido a `board_row_is_full` (Ejercicio 5), pero aquí no puedes cortar el bucle en cuanto encuentras una vacía: tienes que recorrer las `BOARD_WIDTH` celdas enteras para contarlas todas.

### Recuerda

Asume que `board_cell_address` (Ejercicio 3) ya existe y funciona, y úsala con columna=0 para obtener el inicio de la fila.

*board_count_filled_in_row*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Counts how many cells of a row are filled.
;;
;; 📥 INPUT:
;;    B: row
;; 🔙 RETURNS:
;;    A: number of filled cells (0..BOARD_WIDTH)
;;
board_count_filled_in_row::
    ;;; <<YOUR CODE>>
```

## Ejercicio 9

**Implementa** `board_copy_row_down`

### Desplazar una fila hacia abajo

Copia el contenido de una fila a la fila justo debajo (fila+1). Se usa para desplazar el tablero hacia abajo tras borrar una fila llena.

**ⓘ Recuerda**

Asume que `board_cell_address` (Ejercicio 3) ya existe y funciona, y úsala con columna=0 para el inicio de la fila origen. La dirección de la fila siguiente es esa misma dirección `+ BOARD_WIDTH`.

*board_copy_row_down*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies a row into the row immediately
;; below it (row -> row+1).
;;
;; 📥 INPUT:
;;    B: source row
;;
board_copy_row_down::
    ;;; <<YOUR CODE>>
```

## Ejercicio 10

**ⓘ Implementa** `board_column_has_block_above`

**ⓘ Recorrer una columna**

Recorre una columna comprobando varias filas (0, 1, 2...), ***no celdas consecutivas***; el paso entre una fila y la siguiente en la misma columna es `BOARD_WIDTH`, no 1.

**Cuidado con el orden de los registros**

Usa `board_cell_address`, pero ojo: `board_cell_address` espera `B=fila, C=columna`; aquí tu entrada es al revés ( `B=columna` ).

Es fácil equivocarse y llamarla directamente con los registros "tal cual llegan".

*board_column_has_block_above*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether any cell is filled in a
;; column, looking from row 0 up to the given
;; row (not included).
;;
;; 📥 INPUT:
;;    B: column
;;    D: number of rows to check (starting at row 0)
;; 🔙 RETURNS (in the flag, not a register):
;;    NZ, if any cell in that column is filled
;;    Z,  if none is
;;
board_column_has_block_above::
    ;;; <<YOUR CODE>>
```

## Ejercicio 11

**Implementa** `piece_move_left` **y** `piece_move_right`

**Mover una pieza ±1 columna con límites**

Mueve ±1 la columna de la pieza indicada por `hl`, ***sólo si*** no se exceden los límites. Si no, no mueve nada.

¡Ojo! No son celdas de tablero; `hl` apunta directamente a un objeto Pieza, que tiene `piece_row` y `piece_col` seguidos.

Concretamente `hl` tiene la dirección exacta (apunta a) `piece_col`, es decir, al byte que identifica la columna.

**Cuidado con el límite derecho**

El límite izquierdo es 0, el derecho es `BOARD_WIDTH-1`; **no** `BOARD_WIDTH`. Si la columna máxima válida es 7 (con `BOARD_WIDTH=8`), comparar contra `BOARD_WIDTH` dejaría "colarse" una columna de más.

**⚠ Recuerda**

En C, ampersand ( `&` ) es el ***operador dirección***. Aplicado a un objeto ( `&piece_col` ) produce como resultado la dirección de memoria de ese objeto: `&piece_col` → "dirección de memoria de `piece_col` "

*piece_move_left / piece_move_right*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Decrements the column pointed to by HL,
;; without going below 0.
;; 📥 INPUT: HL = address of the column to move (e.g. &piece_col)
piece_move_left::
    ;;; <<YOUR CODE>>

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Increments the column pointed to by HL,
;; without exceeding BOARD_WIDTH-1.
;; 📥 INPUT: HL = address of the column to move (e.g. &piece_col)
piece_move_right::
    ;;; <<YOUR CODE>>
```

## Ejercicio 12

**? Implementa** `piece_move_from_joypad`

**ⓘ Mover la pieza con el D-Pad**

Lee el pad (D-Pad) y, según el botón, llama a `piece_move_left` / `piece_move_right` (Ejercicio 11) con `HL=&piece_col` .

**ⓘ Recuerda**

Estabiliza la lectura leyendo 3 veces, como en el Challenge 2. Esto se hace porque el dispositivo del Pad necesita varios ciclos de reloj para que leamos de él una respuesta correcta (es analógico).

**`bit` no se limita a `A`**

`bit n, r` funciona sobre cualquier registro, no solo `A` — si guardas la lectura del pad en otro registro antes de las llamadas condicionales, evitas que se sobreescriba. Esto importa porque `piece_move_left` / `piece_move_right` sí modifican `A` internamente: si guardaras la lectura del pad en `A`, la perderías al llamar a la primera de las dos.

*piece_move_from_joypad*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Reads the D-Pad and moves piece_col by
;; calling piece_move_left/piece_move_right as needed.
;;
piece_move_from_joypad::
    ;;; <<YOUR CODE>>
```

## Ejercicio 13

**Implementa `board_copy_row_to_tilemap`**

**Copiar una fila al tilemap: renderizar**

Copia una fila del tablero ( `BOARD_WIDTH` celdas) directamente al tilemap. Como `board` guarda IDs de tile (con 0 = tile vacío), sólo tienes que copiar la fila byte a byte en su sitio correcto.

**Deja claro el contrato de registros**

Deja explícito qué registros usas y en qué quedan `DE` / `HL` al terminar — los Ejercicios 14 y 15 van a llamarte una vez por fila, y necesitan saber exactamente eso para no pisarse con lo que les haga falta a ellos.

**⚠ VBLANK / HBLANK**

Esta rutina sólo funciona correctamente si se ejecuta durante VBLANK, HBLANK o con la pantalla apagada.

Asume que será así por simplicidad y no te preocupes.

Después, si quieres probar esto en hardware real, acuérdate de añadir las esperas a VBLANK antes de copiar a VRAM.

*board_copy_row_to_tilemap*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Copies a row of BOARD_WIDTH cells from the
;; board to the tilemap.
;;
;; 📥 INPUT:
;;    DE: source address in board (start of a row)
;;    HL: destination address in the tilemap (start of that row)
;; 🔙 On return: DE and HL have advanced BOARD_WIDTH positions
;;    each (they end up right at the end of the copied row)
;; ❌ Must not use C (Exercises 14 and 15 need it free)
;;
board_copy_row_to_tilemap::
    ;;; <<YOUR CODE>>
```

## Ejercicio 14

**Implementa** `board_render_to_tilemap`

**Recorrer el tablero fila a fila**

Usa `board_copy_row_to_tilemap` (Ejercicio 13): recibe `DE=origen en board, HL=destino en tilemap`, copia una fila, y al volver deja `DE` / `HL` avanzados `BOARD_WIDTH` posiciones cada uno (justo al final de la fila copiada) — sin tocar `C`. Llámala una vez por fila.

**Saltar al inicio de la fila siguiente**

El tilemap tiene 32 bytes por fila, pero el tablero solo `BOARD_WIDTH`. Puedes definirte una constante:

```
DEF NEXT_ROW_OFF = 32 - BOARD_WIDTH
```

y así saltar `NEXT_ROW_OFF` que son las posiciones que le sobran al tilemap para llegar al inicio de la fila siguiente, cuando has completado una fila del tablero.

**Técnica: comparar contra el puntero de fin**

Puedes terminar un bucle sin usar un contador. Piensa que usas `hl` para apuntar a memoria, a un array, por ejemplo (también valdrían `de` o `bc`). Conforme incrementas `hl` puedes compararlo con la dirección donde termina el array. Fíjate en "Código común": hay una etiqueta `board_end:` puesta justo después de la última celda del tablero (antes de `board_sentinel`), que no reserva ningún byte.

`board_end` es, por tanto, la dirección justo después de la última celda. Si `hl` coincide con `board_end`, entonces hemos terminado.

Para hacer la comparación, el truco es comparar primero `l` con la parte baja de `board_end` y luego `h` con la parte alta. No se puede comparar todo junto. Si ambas coinciden, entonces `hl == board_end`.

Esta es la técnica que usan los iteradores de la STL de C++, y muchos lenguajes. Es una alternativa útil a contar iteraciones, y te deja otros registros libres (como `bc` que es práctico para hacer `add hl, bc`).

**Recuerda**

`HIGH()` y `LOW()` son operadores de RGBDS que devuelven el byte alto y el byte bajo de una dirección de 16 bits: `HIGH(board_end)` y `LOW(board_end)`. Te sirven para comparar cada byte del puntero por separado contra `board_end`.

*board_render_to_tilemap*

```
DEF TILEMAP_1_START = $9800
DEF TILEMAP_WIDTH   = 32

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Walks the whole board row by row and copies
;; it to the tilemap, calling
;; board_copy_row_to_tilemap for each one.
;;
board_render_to_tilemap::
    ;;; <<YOUR CODE>>
```

## Ejercicio 15

**Implementa** `board_render_to_tilemap_sentinel`

**Misma tarea, otra técnica de parada**

Exactamente la misma tarea que el Ejercicio 14, pero con otra técnica para saber cuándo parar.

**Técnica: centinela de fin de array**

En vez de comparar el puntero contra `board_end`, hay un centinela (`BOARD_SENTINEL = $FF`, definido en "***Código común***", inicializado en el Ejercicio 2) justo después de la última celda de `board`. Antes de copiar cada fila, comprueba si su primer byte es el centinela: si lo es, ya has terminado.

**Compara las dos técnicas**

Al acabar, compara las dos versiones: ¿cuál te ha costado menos escribir? ¿cuál usa menos instrucciones? Las dos técnicas son útiles: a veces te convendrá una y a veces otra. ¡Tenlas a mano!

*board_render_to_tilemap_sentinel*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Walks the whole board row by row and copies
;; it to the tilemap, stopping when it finds
;; BOARD_SENTINEL instead of comparing pointers.
;;
board_render_to_tilemap_sentinel::
    ;;; <<YOUR CODE>>
```

## Ejercicio 16

**Implementa** `board_piece_fits_bounds`

**Saber si una pieza cabe en el board (sin salirse)**

Rutina genérica: dado un origen `(fila, columna)` y un tamaño `(ancho, alto)`, comprueba si el rectángulo que ocupa esa pieza (su ***bounding box***) cabe **geométricamente** dentro del tablero: sin mirar todavía si hay bloques ahí.

**Condición para caber**

El tablero empieza en (0, 0) y las posiciones (fila, columna) no pueden ser negativas. Por tanto, una ficha cabe si se cumplen a la vez estas dos condiciones:

1. `columna + ancho <= BOARD_WIDTH`
2. `fila + alto <= BOARD_HEIGHT`

Como es un tilemap, y son números enteros, podemos usar estas 2 condiciones que serán más fáciles de implementar:

1. `columna + ancho < BOARD_WIDTH + 1`
2. `fila + alto < BOARD_HEIGHT + 1`

**Cuidado con el caso límite**

Supon el caso `columna + ancho == BOARD_WIDTH + 1` : justo cuando la ficha se sale por la derecha 1 unidad.

Si usas `cp` y saltas directamente con el flag de acarreo (por ejemplo `ret nc`), ten cuidado: la comparación da igualdad exacta (la resta es 0), y eso pone `Z` y `NC` a la vez; estarías devolviendo el flag equivocado sin querer (`Z` en vez de `NZ`).

**ⓘ Por qué es genérica**

Esta rutina es intencionadamente genérica (sirve para piezas de cualquier ancho/alto, no solo de 1x2) — el Ejercicio 17 la va a reutilizar para una pieza con forma de verdad.

*board_piece_fits_bounds*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a rectangular piece fits
;; within the board's bounds (without checking
;; for blocks).
;;
;; 📥 INPUT:
;;    B: origin row (top-left corner)
;;    C: origin column
;;    D: piece width
;;    E: piece height
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if it fits within the bounds
;;    NZ, if it goes outside the board
;;
board_piece_fits_bounds::
    ;;; <<YOUR CODE>>
```

## Ejercicio 17

**? Implementa** `board_check_z_piece_fits`

**ⓘ Forma de la pieza en Z (horizontal)**

La pieza en Z ocupa 4 celdas en un cuadro delimitador de 3 columnas × 2 filas, así

```
|##  |
|  ##|
```

**Reutiliza la comprobación de límites**

Usa `board_piece_fits_bounds` (Ejercicio 16, `B=fila,C=columna,D=ancho,E=alto → Z=cabe,NZ=se sale`) con ancho=3, alto=2 para la comprobación de límites: si no cabe el cuadro delimitador, ya no cabe la pieza.

**Una sola llamada a board_cell_address**

Para las 4 celdas solo necesitas ***una*** llamada a `board_cell_address` (Ejercicio 3), para `(fila,columna)` — el resto de direcciones se obtienen sumando desplazamientos fijos desde esa (+1 para la de al lado, `+BOARD_WIDTH` para bajar una fila).

Si intentas 2 llamadas, tendrás problemas con registros que se sobreescriben. Puedes intentarlo y comprobarlo como ejercicio para aprender más: verás cómo se complica.

*board_check_z_piece_fits*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a Z-piece fits: inside the
;; board and its 4 cells empty.
;;
;;   Piece shape (3x2 bounding box):
;;     X X .
;;     . X X
;;
;; 📥 INPUT:
;;    B: origin row (top-left corner of the 3x2 box)
;;    C: origin column
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if it fits
;;    NZ, if it does NOT fit (goes outside the board or
blocked)
;;
board_check_z_piece_fits::
    ;;; <<YOUR CODE>>
```

## Ejercicio 18

**Implementa** `piece_fits_bounds_check`

**Leer tamaño y posición de la pieza**

Una pieza con el formato `X,Y,W,H,I,...,0` (***Código común***) ya trae su posición y tamaño incluidos en los primeros 4 bytes: no hace falta tocar los incrementos para esta comprobación, solo leer `X,Y,W,H` y llamar a `board_piece_fits_bounds`.

**Recuerda**

Usa `board_piece_fits_bounds` (Ejercicio 16, `B=fila,C=columna,D=ancho,E=alto → Z=cabe,NZ=se sale`). Aquí `fila=Y`, `columna=X`.

*piece_fits_bounds_check*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether a piece fits by size, reading
;; its X,Y,W,H directly from the piece data.
;;
;; 📥 INPUT:
;;    HL: address of the piece (starts at X)
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if it fits by size
;;    NZ, if it does NOT fit
;;
piece_fits_bounds_check::
    ;;; <<YOUR CODE>>
```

## Ejercicio 19

**Implementa** `hl_move_by_increment`

**Primitiva de movimiento genérica**

Rutina genérica y reutilizable.

Dado un incremento `I` (formato `%1111DULR`), cambia el valor de `hl` un paso en la dirección que dice `I`. La rutina no sabe nada de piezas ni de tableros, solo cambia el valor de `hl` según los 4 bits de dirección.

**No hace falta *else* (*switch*)**

Como cada `I` tiene **un solo** bit de dirección a 0 (lógica negativa), puedes hacer 4 comprobaciones seguidas, una por bit, y sólo una actuará de verdad. No hace falta construcciones `if-else` ni saltar entre ellas: sólo una ristra de 'comprobar bit 1, salto condicional, comprobar bit 2, salto condicional, ....'.

**No existe resta de 16 bits**

La Game Boy solo tiene `add hl, rr` para sumar 16 bits — no hay `sub hl, rr`. Para mover "hacia arriba" (restar `BOARD_WIDTH`), suma su negativo:

```
ld bc, -BOARD_WIDTH
add hl, bc
```

*hl_move_by_increment*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Moves HL one step in the direction given
;; by an increment.
;;
;; 📥 INPUT:
;;    HL: current position
;;    A: increment (%1111DULR, negative logic)
;; 🔙 RETURNS:
;;    HL: moved one step in the given direction
;; ❌ Modifies BC (used to shift by a row)
;;
hl_move_by_increment::
    ;;; <<YOUR CODE>>
```

## Ejercicio 20

**Implementa** `piece_fits_no_bounds`

**Ya cabe: falta comprobar ocupación**

Asume que la pieza **ya cabe** dentro de los límites del tablero (eso ya se comprobó en el Ejercicio 18) y comprueba que las celdas estén vacías, recorriendo los incrementos con `hl_move_by_increment` (Ejercicio 19, `HL=posición,A=incremento → HL movido`; modifica `bc`).

**Dos punteros vivos a la vez**

Vas a necesitar dos punteros vivos a la vez: uno para recorrer la pieza (los incrementos) y otro para el tablero. Usa `board_cell_address` (Ejercicio 3), pero ten en cuenta que pisa `hl`. Guarda el valor de `hl` antes de llamarla (mejor en `de` que no es modificado por `board_cell_address` ni `hl_move_by_increment`).

*piece_fits_no_bounds*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Checks whether all cells of a piece are
;; empty (assumes it already fits on the board).
;;
;; 📥 INPUT:
;;    HL: address of the piece (starts at X)
;; 🔙 RETURNS (in the flag, not a register):
;;    Z,  if all cells are empty
;;    NZ, if any is filled
;;
piece_fits_no_bounds::
    ;;; <<YOUR CODE>>
```

## Ejercicio 21

**Implementa** `piece_draw_no_bounds`

**Igual que el Ejercicio 20, pero dibujando**

En vez de comprobar, escribe `1` en cada celda — misma estructura de bucle. Usa `hl_move_by_increment` (Ejercicio 19, modifica `bc`) y `board_cell_address` (Ejercicio 3) para la primera celda.

*piece_draw_no_bounds*

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; Draws a piece on the board (writes 1 into
;; each of its cells). Assumes it already fits.
;;
;; 📥 INPUT:
;;    HL: address of the piece (starts at X)
;;
piece_draw_no_bounds::
    ;;; <<YOUR CODE>>
```